第 17 章：迭代器、生成器与惰性数据流

一句话先说结论：**Python 的迭代器是一条保存了当前位置的数据通道，生成器则是编写这种通道最省力的方式；数据通常不是创建通道时全部产生，而是消费者每调用一次 `next()` 才向上游要一项。**

这个机制很适合处理逐步到达或数量很大的数据，例如：

- 逐行读取日志和 JSONL 文件；
- 按页拉取数据库或 HTTP 查询结果；
- 一边过滤、转换，一边只取前 N 条；
- 接收模型输出片段，为后续异步流打基础。

学完本章，你应该能够解释：

- iterable（可迭代对象）与 iterator（迭代器）有什么区别；
- `for` 如何使用 `iter()` 、 `next()` 和 `StopIteration` ；
- 为什么 list 通常可以重复遍历，同一个生成器却通常只能消费一次；
- `yield` 在哪里暂停，下一次消费又从哪里继续；
- 列表推导式和生成器表达式分别在什么时候计算；
- 惰性流水线何时真的节约内存，何时只是把计算推迟；
- 中途异常或提前停止后，部分结果和打开的资源该怎样处理。

17.1 先建立 Java → Python 心智模型

Python 和 Java 都有迭代协议，但不要只按语法做一一替换：

Java 中熟悉的概念	Python 中可以先类比为	相似点	不能等同的地方
`Iterable<T>`	`Iterable[T]` / iterable	都表示“可以取得遍历能力”	Python 主要靠 `iter(obj)` 协议，不要求显式实现某个接口
`Iterator<T>`	`Iterator[T]` / iterator	都保存当前位置，按次返回元素	Python 没有 `hasNext()` ；耗尽由 `StopIteration` 表示
`Stream<T>`	生成器表达式		Python 生成器本身就是一次性

`Stream<T>` 惰性中间操作	生成器表达式、 生成器函数组成的流水线	终端消费会逐项拉动上游	Python 生成器本身就是一次性 iterator，没有 Java Stream 那套完整操作和并行模型
手写状态机式 `Iterator`	含 `yield` 的生成器函数	都能保存遍历状态	`yield` 由解释器保存局部变量和执行位置，不必手写状态字段与 `next()` 分支
`stream.toList()`	`list(iterator)`	都会把剩余结果物化到内存	Python 的同一个 iterator 会因此耗尽，之后不能自动重放
`try-with-resources`	`with`，以及必要时显式 `close()`	都解决确定性清理	生成器暂停在 `with` 内部时，资源也可能保持打开，直到耗尽或关闭

同步生成器不能类比成 `CompletableFuture`、Reactor `Flux` 或 Reactive Streams `Publisher`。它在调用 `next()` 的线程里同步执行：如果内部进行了阻塞 HTTP 请求，当前线程照样阻塞。惰性只回答“何时计算”，不自动提供异步、并行或响应式背压。

17.2 iterable 提供入口，iterator 保存位置

先把两个术语分开：

- **iterable（可迭代对象）**：`iter(obj)` 能取得一个 iterator；
- **iterator（迭代器）**：`next(iterator)` 能取得下一项，并在对象内部保存当前位置。

list 是典型的 iterable 容器。数据保存在 list 中，每次 `iter(names)` 通常都会创建一个新的 iterator：

```
names = ["Ada", "Lin"]

first = iter(names)
second = iter(names)

print(next(first))  # Ada
print(next(second))  # Ada: second 有自己的位置
```

iterator 自己也是 iterable，而且 `iter(iterator)` 通常返回它自己。因此两个变量如果引用同一个 iterator，它们共享的不是数据副本，而是同一个位置：

README × | readme × | README × | README × | RE
rs/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-0c9114904cebd8a7d1406fec04d8.html
的需求相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单

```
iterator = iter([10, 20, 30])
alias = iterator

print(next(alias))  # 10
print(list(iterator))  # [20, 30]
print(list(alias))  # []：同一个对象已经耗尽
```

所以，**“能用于 `for` ”不等于“能重复遍历”**。list、tuple 等容器通常可以反复创建新 iterator；文件对象、生成器对象和大多数 iterator 则代表一条不断向前的消费路径。

`for` 循环实际依赖什么

下面的 `for`：

```
for item in values:
    use(item)
```

核心语义可以近似展开为：

```
iterator = iter(values)

while True:
    try:
        item = next(iterator)
    except StopIteration:
        break
    use(item)
```

运行过程是：

1. `iter(values)` 请求一个 iterator；
2. 每轮通过 `next(iterator)` 请求一项；
3. 有值时执行循环体；
4. iterator 抛出 `StopIteration` 时，`for` 把它当作正常结束；
5. `ValueError`、`OSError` 等其他异常不会被当成“遍历结束”，而是继续向外传播。

一个正确的 iterator 一旦耗尽，后续调用 `next()` 应继续抛 `StopIteration`，不会自动回到开头。
手动探测时也可以提供默认值：

```
missing = object()
item = next(iterator, missing)

if item is missing:
    print("已经耗尽")
```

这里使用唯一哨兵对象，而不是 `None`，因为 `None` 也可能是合法数据。迭代协议的正式定义可见 Python 3.14 迭代器类型。

需要第二次遍历时怎么办

根据数据来源选择：

1. 原始对象是 list、tuple 等容器：重新调用 `iter(container)`；
2. 数据量小，而且确实需要随机访问或重复使用：在明确的位置物化为 list；
3. 数据能重新读取或查询：保存一个“创建新 iterator”的函数，每次重新调用；
4. 数据源本身不可重放：把一次性语义写进 API 和业务流程，不要假装能够重来。

不要把一个全局生成器对象反复返回。返回的仍是同一个已经前进、甚至已经耗尽的对象。

17.3 生成器：用 `yield` 写一个可暂停的函数

函数体中出现 `yield` 后，它就成为**生成器函数**。调用生成器函数只创建一个生成器对象，不会立刻从头执行整个函数体：

```
from collections.abc import Iterator


def count_up_to(limit: int) -> Iterator[int]:
    current = 1
    print("函数体开始")
    while current <= limit:
        print("生产", current)
        yield current
        current += 1


numbers = count_up_to(2)  # 没有打印：只创建对象
print(next(numbers))  # 打印“函数体开始”“生产 1”，然后得到 1
```

```
numbers = count_up_to(2)  # 没有打印：只创建对象
print(next(numbers))  # 打印“函数体开始”“生产 1”，然后得到 1
print(next(numbers))  # 从 yield 后恢复，current 变成 2，然后得到 2
```

每次执行 `yield value`，生成器会：

1. 把 `value` 返回给当前这次 `next()`；
2. 暂停在 `yield` 所在位置；
3. 保存局部变量、执行位置和当前异常处理状态；
4. 等下一次 `next()`，再从 `yield` 后面继续。

可以先把它理解成“解释器替你生成了一个状态机”，但不能理解成后台线程。生成器暂停期间没有在后台继续工作，也不会预先生产下一项。

`yield`、`return` 和 `yield from`

写法	对当前生成器的作用	消费者看到什么
`yield item`	产出一项并暂停	当前 `next()` 返回 `item`
`return` 或运行到函数末尾	结束生成器	下一次请求收到 `StopIteration`
`return value`	结束并携带一个完成值	普通 `for` 看不到；手动捕获的 `StopIteration.value` 可以看到
`yield from source`	逐项委托给另一个 iterable	像连续 `yield` 该来源的每一项

组合多个来源时，`yield from` 能避免手写重复的转发循环：

```
def combined() -> Iterator[str]:
    yield "header"
    yield from ("local-A", "local-B")
    yield from ("cache-C",)


print(list(combined()))
# ['header', 'local-A', 'local-B', 'cache-C']
```

`yield from` 不会自动把子来源先转成 list。它还支持更完整的生成器委托协议，但普通单向数据流先掌握"逐项转交"即可。详细语义见
Python 3.14 `yield` 表达式。

生成器内部也不要用 `raise StopIteration` 表示正常结束。应写 `return` 或让函数自然结束；从生成器函数内部意外逸出的 `StopIteration` 会被转换成 `RuntimeError`，避免内部错误被外层 `for` 误判成正常耗尽。

`list(iterator)` 是一次完整消费

`list(iterator)` 会反复调用 `next()`，直到 iterator 耗尽，并把所有结果保存下来。调用结束后：

- 剩余元素已经全部生产；
- 中途副作用已经发生；
- iterator 已经耗尽；
- 返回的 list 占用保存全部结果引用所需的内存。

因此，`list(...)` 不是更换语法外壳，而是一个明确的**物化边界**。测试小数据、获取长度、使用下标或重复遍历时，它很合适；面对无界流或只需要前几项的数据时，则要谨慎。

17.4 列表推导式与生成器表达式：只差括号，执行时机不同

```
squares_list = [value * value for value in values]
squares_iterator = (value * value for value in values)
```

问题	列表推导式 `[...]`	生成器表达式 `(...)`
创建时	遍历输入、计算全部结果并创建 list	创建生成器 iterator
元素表达式与过滤条件	创建 list 时执行	消费相应候选项时执行
结果内存	保存全部结果引用	通常只保存暂停状态和当前项
同一个结果能否重复遍历	可以	通常不可以，耗尽即结束
适合	小而有界，需要完整结果	单次流水线、可能很大、可能提前停止

生成器表达式有一个容易漏掉的精确规则：**最左侧 `for` 的 iterable 表达式会立即求值，并立即对结果调用 `iter()`；元素表达式、过滤条件和后续 `for` 才惰性执行。**

```
def prepare() -> list[int]:
    print("准备输入")
    return [1, 2, 3]


squares = (value * value for value in prepare())
# 此时已经打印"准备输入"，但平方还没有计算

print(next(squares))  # 1：现在只计算第一项
```

如果最左侧调用本身是生成器函数，调用它仍然只返回尚未启动的生成器对象，因此其函数体要等消费时才执行。官方的精确求值规则见
Python 3.1.4 生成器表达式。

惰性何时才真的降低峰值内存

一条流水线通常要同时满足以下条件，才能获得流式处理的内存优势：

- 上游可以逐项提供数据，没有先创建巨大的 list；
- 过滤、转换等中间阶段没有偷偷 `list(source)`、全量排序或全量分组；
- 下游逐项处理或提前停止，没有再次收集所有结果；
- 处理完的元素没有被缓存、闭包或日志集合长期引用；
- 单个元素本身不是一个巨型对象。

下面三种写法都可能打破全程惰性：

```
source = load_everything()  # 上游已经全部放进 list
stream = (transform(x) for x in source)

cached = list(stream)  # 中间或末尾重新收集全部结果
sorted_items = sorted(stream)  # 排序通常也必须先看完全部输入
```

这些操作不一定错误。业务需要排序、总数、随机访问或重复遍历时，就应该物化；关键是把这个边界写清楚，不要把"外面套了生成器"描述成"全程流式"。

17.5 拉式流水线：消费者反向驱动上游

本章项目把 `documents.jsonl` 处理成搜索结果：

E README readme README README README
ders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-0c9114904cebd8a7d1406fec04d8.html
明接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

17.5 拉式流水线：消费者反向驱动上游

本章项目把 `documents.jsonl` 处理成搜索结果：

```
数据方向：JSONL 文件 → 校验 → 过滤 → 转换 → 最多 N 条 → 消费者
调用方向：JSONL 文件.← next ← next ← next ← next ← 消费者
```

数据从源头流向消费者，但控制请求从消费者反向传递。消费者要一项，限量器向转换器要一项，转换器向过滤器要一项，过滤器再向文件源要一行。如果某行没有通过过滤，过滤器会继续拉取，直到找到下一条合格记录或上游耗尽。

这种组合方式的价值不只是少占内存：消费者如果拿够两项就停止，上游后续的文件读取、JSON 解析、转换和副作用都不会发生。

限制 N 条时，不要多取第 N+1 条

下面的写法会先从 `source` 取得一项，再执行循环体中的判断：

```
# 反例：index == limit 时，item 已经从上游取出来了。
for index, item in enumerate(source):
    if index >= limit:
        break
    yield item
```

如果“取得一项”意味着发起 HTTP 请求、移动数据库游标或可能触发异常，这次多拉取就有实际成本。可以精确控制 `next()` 次数：

```
from collections.abc import Iterable, Iterator


def take(items: Iterable[str], limit: int) -> Iterator[str]:
    iterator = iter(items)
    for _ in range(limit):
        try:
            yield next(iterator)
        except StopIteration:
            return
```

README × | README × | readme × | README × | README × | README
ders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-0c9114904cebd8a7d1406fec04d8.html
接的需求相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

`limit == 0` 时，上游一次也不会被拉取。工程代码还应明确负数是报错还是按零处理。标准库的 `itertools.islice(items, limit)` 也适合完成这种截取。

外部数据仍要在运行时校验

本章给流水线元素标注了 `Document` 和 `SearchHit`。这些注解帮助 IDE、类型检查器和读者理解每层输入输出，但不会自动验证 `json.loads()` 得到的真实对象。

因此文件源仍要检查：顶层是不是字典、必需字段是否存在、字段值类型是否正确。通过边界校验后，后续生成器才可以诚实地按 `Iterator[Document]` 处理。类型注解不能替代 JSON、HTTP、数据库等外部边界的运行时校验。

17.6 异常发生在消费时，之前的结果不会自动回滚

对生成器函数来说，创建和执行是两个时间点：

```
stream = load_jsonl(path, events)  # 只创建生成器，通常还没有打开文件
first = next(stream)  # 现在才可能发生文件、解码和字段校验错误
```

假设流水线的事件顺序是：

```
产出 doc-001 → 产出 doc-003 → 转换 doc-error 时抛异常
```

前两项已经真实交给消费者。对应的日志、写入、计费等副作用也可能已经发生。生成器不会提供事务语义，无法因为第三项失败而自动撤销前两项。

需要保留部分结果时，应在消费边界逐项保存，并捕获具体异常：

```
partial = []

try:
    for item in pipeline:
        partial.append(item)
except DocumentTransformError as error:
    report(error, partial)
```

如果直接写 `results = list(pipeline)`，异常会阻止赋值完成。`list()` 内部已经收集的临时结果不会通过 `results` 交付给调用方。是否保留部分结果、是否回滚副作用、从哪里重试，都必须由业务层

README | readme | README | README | README
lers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-0c9114904cebd8a7d1406fec04d8.html
的需求相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

如果直接写 `results = list(pipeline)`，异常会阻止赋值完成。`list()` 内部已经收集的临时结果不会通过 `results` 交付给调用方。是否保留部分结果、是否回滚副作用、从哪里重试，都必须由业务层明确决定。

异常从生成器内部传播出去后，这个生成器通常已经终止。不要尝试从同一对象的失败位置继续；需要重试时，应根据数据源是否可重放、操作是否幂等，创建一条新的迭代路径。

17.7 提前停止时，谁负责关闭资源

生成器可以暂停在 `with` 块内部：

```
from collections.abc import Generator
from pathlib import Path


def load_lines(path: Path) -> Generator[str]:
    with path.open(encoding="utf-8") as file:
        for line in file:
            yield line.rstrip("\n")
```

第一次 `next()` 会打开文件并停在 `yield`。如果消费者完整耗尽生成器，控制流会自然离开 `with` 并关闭文件；但消费者也可能 `break`、只取前 N 条，或在下游转换时抛异常。此时文件源可能仍暂停着。

本章示例采用明确的所有权规则：**创建并持有文件源的代码负责最终关闭它。**

```
source = load_jsonl(DATA_PATH, events)
pipeline = build_pipeline(source, limit=2, events=events)

try:
    results = consume_hits(pipeline, events)
finally:
    source.close()
```

`close()` 会在生成器暂停的 `yield` 位置触发 `GeneratorExit`，从而让内部的 `finally` 和已经进入的 `with` 有机会执行清理。已经结束或尚未启动的生成器也可以安全关闭。

不要把"当前使用 CPython"当成资源管理策略，也不要依赖对象何时被垃圾回收。还要注意：关闭某个外层生成器，并不普遍保证它会关闭任意上游 iterable。本章保留真正文件源的引用，并直接关闭资源

README × | README × | readme × | README × | README × | README
wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-0c9114904cebd8a7d1406fec04d8.html
相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

不要把"当前使用 CPython"当成资源管理策略，也不要依赖对象何时被垃圾回收。还要注意：关闭某个外层生成器，并不普遍保证它会关闭任意上游 iterable。本章保留真正文件源的引用，并直接关闭资源所有者。更系统的资源生命周期写法会在第 18 章继续展开。

17.8 类型注解：什么时候写 `Iterable`、`Iterator` 或 `Generator`

最常见的生成器签名如下：

```
from collections.abc import Iterable, Iterator


def normalize(lines: Iterable[str]) -> Iterator[str]:
    for line in lines:
        text = line.strip()
        if text:
            yield text
```

- 参数写 `Iterable[str]`：函数只要求能够取得字符串，不强迫调用方传 list；
- 返回写 `Iterator[str]`：调用方拿到一条保存位置、通常一次性消费的字符串流；
- `Iterator[Document]` 中的 `Document` 表示每次产出的元素类型；
- 注解服务于阅读和静态检查，不会改变运行时求值方式，也不会验证每个元素。

当公共 API 需要明确暴露 `send()`、`throw()`、`close()` 或生成器完成值时，可以写更具体的：

```
Generator[YieldType, SendType, ReturnType]
```

本项目使用 Python 3.14，其中后两个类型参数默认是 `None`，所以 `Generator[Document]` 等价于 `Generator[Document, None, None]`。官方注解说明见 Python 3.14 生成器与协程注解。

同步生成器不是 asyncio coroutine

对象	常见创建方式	谁驱动它	怎样取结果	阻塞 I/O 会怎样
同步生成器	含 `yield` 的 `def`	当前消费者	`next()` / `for`	阻塞当前线程
				只有正确使用异步 I/O

同步生成器不是 asyncio coroutine

对象	常见创建方式	谁驱动它	怎样取结果	阻塞 I/O 会怎样
同步生成器	含 `yield` 的 `def`	当前消费者	`next()` / `for`	阻塞当前线程
coroutine	调用 `async def`	`await` 与事件循环	`await`	只有正确使用异步 I/O 才能让出执行权
异步生成器	含 `yield` 的 `async def`	异步消费者与事件循环	`async for`	取决于内部是否等待异步 I/O

三者都可能暂停，但协议、调度方式和使用场景不同。本章只讨论同步生成器；coroutine 在第 19 章展开，异步流在第 24 章展开。

17.9 Java 工程师最容易踩的坑

1. **把 iterable 当成可重放集合。** iterator 自己也是 iterable；先确认拿到的是容器、工厂，还是一次性数据流。
2. **为了调试执行 `print(list(stream))`，随后再交给业务代码。** 调试语句已经耗尽原对象。
3. **生成器内部先写 `cached = list(source)`。** 这只是把全量读取推迟到第一次消费。
4. **认为生成器表达式创建时绝无副作用。** 最左侧 iterable 表达式及其 `iter()` 会立即执行。
5. **用 `enumerate` 取到元素后才检查上限。** 判断时第 N+1 项已经被拉取。
6. **认为惰性等于异步。** 同步生成器里的阻塞调用仍阻塞当前线程。
7. **认为异常会撤回已经 yield 的结果。** 生成器不是事务，部分结果和副作用已经发生。
8. **只测试完整消费。** 真实消费者会提前停止，也会在某一项中途失败，清理路径必须覆盖这些情况。

选择数据结构时，可以使用下面的判断：

需求	优先选择	原因
数据小，需要长度、下标、排序或重复遍历	list / 列表推导式	结果完整，语义直接
单次、短小的按需计算链	生成器表达式	紧凑，逐项计算
有分支、日志、异常或清理	生成器函数	`yield` 周围能清楚表达控制流
组合多个 iterable	`yield from`	直接逐项委托，不重复写转发循环

README × | README × | readme × | README × | README × | README
llers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-0c9114904cebd8a7d1406fec04d8.html
接的需求相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

公共函数只需要读取输入	接收 `Iterable[T]`	list、tuple 和一次性流都能传入
返回带当前位置的一次性流	返回 `Iterator[T]`	把消费语义写进接口
后续确实需要完整结果	在明确边界 `list(...)`	内存成本和所有权可见

17.10 运行本章项目

本章只使用标准库，不需要安装额外依赖。根 `pyproject.toml` 要求 Python `>=3.14` 。

目录中的文件职责如下：

文件	职责
`README.md`	建立迭代协议、生成器和惰性数据流的完整心智模型
`example.py`	打印六组生产/消费事件，并用断言验证关键边界
`documents.jsonl`	本地、固定、可逐行读取的数据源

从仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/17_iterators_generators/example.py
```

`-B` 只阻止生成 `.pyc` ，不改变运行语义。 `java_to_python/run_all.py` 目前只包含前 10 章，因此不要用它运行本章。

阅读输出时，不必背所有日志，重点验证以下因果关系：

1. 调用生成器函数后事件数仍为 0，第一次 `next()` 才开始生产；
2. 第二次 `next()` 从上一个 `yield` 后恢复，局部变量没有重置；
3. 列表推导式立即计算全部元素，生成器表达式按消费次数计算；
4. 限制两条合格结果时只读取 JSONL 前三行，第四行 `doc-error` 没有被提前拉取；
5. 放宽上限后，转换到 `doc-error` 才失败，之前两条结果已经交付；
6. 提前停止后文件尚未自动关闭，显式 `close()` 后关闭事件出现。

可选的静态检查与格式检查：

```
.venv/bin/ruff check java_to_python/chapters/17_iterators_generators
```

ME README readme README README README
olders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-0c9114904cebd8a7d1406fec04d8.html
期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

可选的静态检查与格式检查：

```
.venv/bin/ruff check java_to_python/chapters/17_iterators_generators
.venv/bin/ruff format —check java_to_python/chapters/17_iterators_generators
.venv/bin/pyright java_to_python/chapters/17_iterators_generators/example.py
```

本地实验能验证同步拉取顺序、逐行读取、过滤转换、限量、部分失败和文件关闭。它没有模拟真实 HTTP 分块、数据库游标、LLM token 边界、网络背压、超时重试或 asyncio 调度；换成真实 I/O 客户端后，这些行为仍需集成测试。

17.11 递进练习

练习 1：观察共享迭代位置

要求：为 `[10, 20, 30]` 创建一个 iterator 和两个别名，交替调用 `next()`，最后用手写 `while / StopIteration` 消费剩余项。

验收标准：

- 运行前能写出每次 `next()` 的结果；
- 能证明两个别名共享位置，而新调用 `iter(list)` 会从头开始；
- 耗尽后分别使用异常和默认值两种方式探测。

练习 2：区分创建时和消费时

要求：编写一个会记录事件的输入构造函数和一个 `trace_double(value)`，分别放入列表推导式和生成器表达式。输入固定为 `[1, 2, 3, 4]`，生成器表达式只消费两项。

验收标准：

- 列表推导式创建时计算四项；
- 生成器表达式只计算两项；
- 能解释为什么生成器表达式创建时仍出现了最左侧输入构造事件。

练习 3：实现不多拉一项的 take

要求：实现 `take(source, limit)`，并用可计数的数据源分别测试 `limit=0` 、 `2` 、 `10` 。数据源最多产出五项，不允许内部先 `list(source)` 。

练习 3：实现不多拉一项的 take

要求：实现 `take(source, limit)`，并用可计数的数据源分别测试 `limit=0` 、 `2` 、 `10` 。数据源最多产出五项，不允许内部先 `list(source)` 。

验收标准：

- `limit=0` 时上游生产 0 次；
- `limit=2` 时上游恰好生产 2 次；
- `limit=10` 时得到全部 5 项并自然耗尽；
- 明确负数参数在创建时还是首次消费时失败，并解释原因。

练习 4：处理 JSONL 的部分失败

要求：复制一份 JSONL 到临时目录，先放两行合法记录，再放一行非法 JSON，最后再放一行合法记录。逐项收集成功结果，捕获具体错误，并确保所有路径都关闭文件。

验收标准：

- 错误只在消费到非法行时发生；
- 能看到错误前已经交付的两项；
- 不声称错误后的记录已经处理；
- 正常耗尽、中途异常、提前停止都能观察到确定的关闭证据。

练习 5：模拟模型文本片段

要求：用同步生成器依次产出 `"你"` 、 `"好"` 、 `"!"` ，消费者在两项后停止，并分别记录生产和消费事件。

验收标准：

- 创建生成器时没有生产事件；
- 只消费两项就不会生产第三项；
- 明确说明该替身只验证同步迭代和提前停止，不验证真实模型、网络传输、token 切分或异步行为。

17.12 本章小结

- iterable 负责提供 iterator，iterator 负责保存当前位置并逐项返回数据；
- `for` 会反复调用 `next()` ，并把 `StopIteration` 当作正常结束；

- `limit=10` 时得到全部 5 项并自然耗尽；
- 明确负数参数在创建时还是首次消费时失败，并解释原因。

练习 4：处理 JSONL 的部分失败

要求：复制一份 JSONL 到临时目录，先放两行合法记录，再放一行非法 JSON，最后再放一行合法记录。逐项收集成功结果，捕获具体错误，并确保所有路径都关闭文件。

验收标准：

- 错误只在消费到非法行时发生；
- 能看到错误前已经交付的两项；
- 不声称错误后的记录已经处理；
- 正常耗尽、中途异常、提前停止都能观察到确定的关闭证据。

练习 5：模拟模型文本片段

要求：用同步生成器依次产出 `"你"`、`"好"`、`"!"`，消费者在两项后停止，并分别记录生产和消费事件。

验收标准：

- 创建生成器时没有生产事件；
- 只消费两项就不会生产第三项；
- 明确说明该替身只验证同步迭代和提前停止，不验证真实模型、网络传输、token 切分或异步行为。

17.12 本章小结

- iterable 负责提供 iterator，iterator 负责保存当前位置并逐项返回数据；
- `for` 会反复调用 `next()`，并把 `StopIteration` 当作正常结束；
- 生成器函数调用时只创建对象，`yield` 在消费时产出一项、暂停并保存状态；
- 同一个 iterator 通常只能消费一次，需要重放时必须创建新 iterator 或明确物化；
- 生成器表达式大部分逻辑惰性执行，但最左侧 iterable 表达式会立即求值；
- 真正的低峰值内存依赖上游、中间阶段和消费者共同保持逐项处理；
- 消费阶段可能已经产生部分结果，异常不会自动回滚；
- 提前停止时资源可能仍处于打开状态，必须建立明确的关闭责任；
- 同步生成器只是同步拉式迭代，不会自动变成 asyncio、并行计算或响应式流。

```jsonl
{"document_id":"doc-001","title":"生成器基础","category":"python","score":0.95}
{"document_id":"doc-002","title":"普通数据库记录","category":"database","score":0.92}
{"document_id":"doc-003","title":"惰性文档流水线","category":"python","score":0.88}
{"document_id":"doc-error","title":"受控转换失败","category":"python","score":0.86}
{"document_id":"doc-005","title":"未被提前读取的文档","category":"python","score":0.82}
```

```python
"""第 17 章：迭代器、生成器与惰性数据流。

每一段都打印生产与消费事件。示例只访问同目录的受控 JSONL 数据，不访问网络，
也不要求 API Key。
"""

# 建议按 main() 中的六个 demonstrate_* 函数依次阅读：先理解迭代协议和生成器暂停，
# 再看推导式、yield from，最后把这些规则放进 JSONL 拉式流水线。
# 流水线的数据方向是“文件 -> 过滤 -> 转换 -> 限量 -> 消费者”；真正执行时则由消费者调用
# next()，一层层向上游索要下一项。events 会同时记录两种方向上的事件，方便核对先后顺序。

# json.loads() 把文件中的一行 JSON 文本解析成 Python 对象；此时对象结构仍不可信，
# 后面还要交给 parse_document() 做运行时边界校验。
import json

# 这些泛型描述函数需要或返回的迭代能力。它们主要服务于阅读和静态类型检查，
# 不会在运行时替我们验证每个元素的真实类型。
from collections.abc import Generator, Iterable, Iterator

# Path 用对象表示路径；相比手工拼接字符串，它能直接提供 open()、name 等路径操作。
from pathlib import Path

# TypedDict 用于描述“具有固定键和值类型的字典”，但运行时得到的仍是普通 dict。
from typing import TypedDict

# __file__ 是当前源码文件的路径；with_name() 只替换文件名，所以无论从哪个工作目录启动，
# 都会定位到与 example.py 同目录的 documents.jsonl。
DATA_PATH: Path = Path(__file__).with_name("documents.jsonl")
# 关闭动作会以这条固定消息写入 events。后面的检查用成员测试确认清理路径确实执行过。
CLOSED_EVENT = "源：文件已关闭"


# Document 是 JSONL 数据通过校验后在流水线内部使用的结构约定。
# 它不会像 Java DTO 那样创建 Document 实例：值仍是 dict，通过 document["title"] 访问字段；
# 这里列出的键只让编辑器、类型检查器和读者知道流水线可以依赖哪些内容。
class Document(TypedDict):
    """从 JSONL 边界校验后、流水线内部使用的文档结构。"""
```

```python
class Document(TypedDict):
    """从 JSONL 边界校验后，流水线内部使用的文档结构。"""

    document_id: str
    title: str
    category: str
    score: float


# SearchHit 是转换后的精简字典，只保留消费者真正需要的标识和展示文本。
# 把输入、输出定义成两个 TypedDict，可以让转换前后的字段边界在类型提示中保持清晰。
class SearchHit(TypedDict):
    """转换阶段交给消费者的精简结果。"""

    document_id: str
    label: str


# 为输入格式错误定义专门异常，调用方就能把“数据不符合约定”与其他 ValueError 场景区分开。
# 继承 ValueError 表明错误值已经存在，但内容或类型不满足当前数据边界。
class DocumentFormatError(ValueError):
    """某行 JSON 的字段不满足本地文档约定。"""


# 这个异常只服务于中途失败实验，用来证明：某一项转换失败前，先前 yield 的结果已经交付。
class DocumentTransformError(RuntimeError):
    """教学用的可控转换失败。"""


# -> None 表示函数只产生打印副作用，没有有意义的返回值。
# f-string 中的 {title} 会在调用时求值并嵌入字符串；开头的 \n 先输出一个空行分隔实验。
def print_section(title: str) -> None:
    print(f"\n=== {title} ===")


# events 是调用方传入的可变 list。本函数先原地 append，再打印同一条消息；
```

```python
def record(events: list[str], message: str) -> None:
    """同时保存和打印事件，便于断言，也便于肉眼观察真实顺序。"""
    events.append(message)
    print(f"  事件 | {message}")


# Iterable[str] 只要求 values 能产生字符串，不限制它必须是 list；tuple 或生成器也都能传入。
# 返回值则明确物化成 list，便于与真实 for 循环的消费结果比较。
def consume_like_for(values: Iterable[str]) -> list[str]:
    """显式写出 for 循环最核心的 iter/next/StopIteration 过程。"""
    # iter(values) 向可迭代对象请求一个迭代器；真正的当前位置保存在 iterator 中。
    iterator: Iterator[str] = iter(values)
    consumed: list[str] = []

    # Python 的 for 会替我们重复 next() 并把 StopIteration 当作正常结束。
    # 这里手写同样的协议；故意用无限循环，把“何时停止”交给迭代器决定。
    while True:
        try:
            # 每次 next() 只索取一项；若有值，item 就绑定到这次返回的字符串。
            item: str = next(iterator)
        except StopIteration:
            # 只捕获协议规定的耗尽信号。生产过程中的其他异常仍应向调用方传播。
            print("  手动消费者收到 StopIteration，循环结束")
            break
        # 只有成功取得值才会到这里；先观察，再把该值保存到最终的急切结果中。
        print(f"  手动消费者拿到：{item}")
        consumed.append(item)
    # while 结束说明输入已耗尽，返回的新 list 保存了所有已经消费的字符串引用。
    return consumed


# 只要函数体里出现 yield，它就是生成器函数。调用 counted_source(...) 只创建生成器对象，
# 函数体要等第一次 next() 才开始；Iterator[int] 表示它随后会逐项产出整数。
def counted_source(limit: int, events: list[str]) -> Iterator[int]:
    """每次只生产一个整数，并让被保留的局部状态可以观察。"""
    produced = 0
    record(events, f"生成器函数体开始，produced={produced}")
```

```python
def counted_source(limit: int, events: list[str]) -> Iterator[int]:
    """每次只生产一个整数，并让被保留的局部状态可以观察。"""
    produced = 0
    record(events, f"生成器函数体开始, produced={produced}")

    # produced 同时是产出数量和下一次恢复时要继续使用的局部状态。
    while produced < limit:
        produced += 1
        record(events, f"生产第 {produced} 个值")
        # yield 把当前整数交给这次 next() 的调用者，并暂停整个函数，而不是退出函数。
        yield produced
        # 只有调用方再次请求值，控制流才会从上一个 yield 后面恢复；produced 不会重新初始化。
        record(events, f"从 yield 后恢复, produced 仍为 {produced}")

    # 函数自然结束或执行不带值的 return，都会让下一次 next() 收到 StopIteration。
    record(events, "执行 return, 后续 next 将得到 StopIteration")


# 这是普通函数而非生成器函数，所以调用时会立即执行函数体并返回完整 list。
# 第 3 个实验把它放在生成器表达式最左侧，用事件证明那一小部分会在创建表达式时求值。
def build_eager_iterable(events: list[str]) -> list[int]:
    """普通函数用于暴露生成器表达式最左侧表达式的立即求值。"""
    record(events, "计算最左侧 iterable 表达式并创建 list")
    return [1, 2, 3]


# 把平方计算包装成函数只是为了记录“元素表达式何时真正求值”；返回值仍是普通 int。
def trace_square(value: int, events: list[str]) -> int:
    record(events, f"计算 {value} 的平方")
    return value * value


# Generator[产出类型, send 类型, return 完成值类型] 比 Iterator 能表达更完整的生成器协议。
# 这里逐项 yield str，不接收 send() 的非 None 值，最终 return 的完成值也是 str。
def one_chunk_then_return(events: list[str]) -> Generator[str, None, str]:
    """return 的字符串不是 yield 出去的数据，而是生成器完成值。"""
```

```python
def one_chunk_then_return(events: list[str]) -> Generator[str, None, str]:
    """return 的字符串不是 yield 出去的数据，而是生成器完成值。"""
    record(events, "准备 yield 一个数据块")
    yield "数据块"
    # 再次 next() 后从上面的 yield 恢复；return 的字符串进入 StopIteration.value，
    # 它不会像又写了一次 yield 那样成为普通遍历结果。
    record(events, "执行 return，结束生成器")
    return "完成值"


# 把任意字符串 iterable 加上来源名前缀再逐项转发。传入 tuple 时仍不会先复制成 list。
def named_source(name: str, values: Iterable[str], events: list[str]) -> Iterator[str]:
    record(events, f"{name} 开始")
    # for 每轮从 values 取一个 value；item 是当前轮新建的格式化字符串。
    for value in values:
        item: str = f"{name}:{value}"
        record(events, f"{name} yield {item}")
        # 每产出一项就暂停，所以下一轮要等下游再次请求时才会发生。
        yield item
    # 走到这里表示 values 已自然耗尽；没有 yield 的 return 只结束本层生成器。
    record(events, f"{name} return")


# 这个生成器展示两种产出方式：yield 自己的一项，yield from 委托另一个 iterable。
def combine_sources(events: list[str]) -> Iterator[str]:
    """yield 负责单个值，yield from 逐项转交两个来源。"""
    record(events, "组合器开始")
    yield "组合器:开头"
    # yield from 会依次转交子生成器的每一项及其暂停过程，不会预先物化整个子来源。
    yield from named_source("本地", ("A", "B"), events)
    yield from named_source("缓存", ("C",), events)
    record(events, "组合器 return")


# raw 标成 object，表示 json.loads() 的边界输出尚不能信任；它可能是 dict、list、数字等。
# 参数列表中的 * 让 line_number 只能按关键字传入，调用处必须明确写出行号的含义。
```

```
173
174 # raw 标成 object，表示 json.loads() 的边界输出尚不能信任；它可能是 dict、list、数字等。
175 # 参数列表中的 * 让 line_number 只能按关键字传入，调用处必须明确写出行号的含义。
176 # 校验通过后返回符合 Document 结构的普通 dict，并统一把 score 转成 float。
177 def parse_document(raw: object, *, line_number: int) -> Document:
178     """做最小运行时校验，避免用类型注解冒充 JSON 边界验证。"""
179     if not isinstance(raw, dict):
180         raise DocumentFormatError(f"第 {line_number} 行必须是 JSON object")
181
182     # dict.get() 在键不存在时返回 None，便于下面把“缺字段”和“值类型错误”统一拒绝。
183     # 此处只提取允许的字段，输入里即使有额外键也不会被带入流水线内部。
184     document_id: Any | None = raw.get("document_id")
185     title: Any | None = raw.get("title")
186     category: Any | None = raw.get("category")
187     score: Any | None = raw.get("score")
188
189     # `or not value` 还会拒绝空字符串；每个分支都带行号，错误能定位回 JSONL 原文。
190     if not isinstance(document_id, str) or not document_id:
191         raise DocumentFormatError(f"第 {line_number} 行 document_id 必须是非空字符串")
192     if not isinstance(title, str) or not title:
193         raise DocumentFormatError(f"第 {line_number} 行 title 必须是非空字符串")
194     if not isinstance(category, str) or not category:
195         raise DocumentFormatError(f"第 {line_number} 行 category 必须是非空字符串")
196     # bool 在 Python 中是 int 的子类，所以 isinstance(True, (int, float)) 为 True；
197     # 业务分数不应接受布尔值，必须先单独排除，再用类型元组同时接受 int 和 float。
198     if isinstance(score, bool) or not isinstance(score, (int, float)):
199         raise DocumentFormatError(f"第 {line_number} 行 score 必须是数字")
200
201     # 新建字典切断了对原始解析结果的依赖；float(score) 让内部数值表示稳定为浮点数。
202     return {
203         "document_id": document_id,
204         "title": title,
205         "category": category,
206         "score": float(score),
207     }
208
```

```python
# Generator[Document] 暴露了 close()，持有者可在提前停止时显式触发 finally。
# 这是拉式文件源：首次消费才打开文件，每恢复一次最多继续读到下一条有效文档。
def load_jsonl(path: Path, events: list[str]) -> Generator[Document]:
    """打开后逐行解析；函数内部没有先 list(...) 缓存所有文档。"""
    # 这行位于生成器函数体内，所以调用 load_jsonl(...) 时还不会执行；
    # 第一次 next(source) 才会从这里开始运行并尝试打开文件。
    record(events, f"源: 准备打开 {path.name}")
    # opened 区分“成功进入 with 后的关闭”与“open() 自身失败”，避免记录不存在的关闭动作。
    opened = False
    try:
        # with 类似 Java try-with-resources；但生成器在内部 yield 时，with 代码块也会一起暂停，
        # 文件会保持打开，直到继续耗尽、异常退出或生成器被显式 close()。
        with path.open("r", encoding="utf-8") as file:
            opened = True
            record(events, "源: 文件已打开")
            # 文件对象本身可迭代，每轮惰性读取一行。enumerate(..., start=1) 同时生成从 1
            # 开始的行号，并将二元组解包到 line_number、line，正好对应编辑器中的行号。
            for line_number, line in enumerate(file, start=1):
                record(events, f"源: 读取第 {line_number} 行")
                # strip() 返回去掉两端空白的新字符串；空行不进入 JSON 解析，直接请求下一行。
                text: str = line.strip()
                if not text:
                    continue

                # loads() 处理的是 str 而不是文件；非法 JSON 会在消费到这一行时直接抛异常。
                raw: object = json.loads(text)
                # 关键字实参让“当前枚举行号传给仅限关键字参数”一目了然。
                document: Document = parse_document(raw, line_number=line_number)
                record(events, f"源: yield {document['document_id']}")
                # 暂停前把一条已校验字典交给过滤器；下一行要等整个下游再次拉取才读取。
                yield document
    finally:
        # 源自然耗尽、源内部在消费时抛异常，或持有者显式 close()，都会关闭文件并经过这里。
        # 如果 open() 自己失败，opened 仍为 False，不能错误声称一个从未打开的文件已被关闭。
        if opened:
            record(events, CLOSED_EVENT)
```

```
212  def load_jsonl(path: Path, events: list[str]) -> Generator[Document]:
241      finally:
242          # 源自然耗尽、源内部在消费时抛异常，或持有者显式 close()，都会关闭文件并经过这里。
243          # 如果 open() 自己失败，opened 仍为 False，不能错误声称一个从未打开的文件已被关闭。
244          if opened:
245              record(events, CLOSED_EVENT)
246
247
248  # 输入只要求 Iterable[Document]，因此既能接 list/tuple，也能直接串接一次性的文件生成器。
249  # 参数列表中的 * 表示后面三项必须使用关键字传参，避免多个字符串、数字实参含义混淆。
250  def filter_documents(
251      documents: Iterable[Document],
252      *,
253      category: str,
254      minimum_score: float,
255      events: list[str],
256  ) -> Iterator[Document]:
257      # 下游每要一项，本层就从 documents 逐条寻找下一条符合两个条件的文档。
258      for document in documents:
259          # and 具有短路语义；这里两侧只是 dict 读取和比较，不会提前消费更多上游数据。
260          accepted: bool = document["category"] == category and document["score"] >= minimum_score
261          # 条件表达式按“为真时的值 if 条件 else 为假时的值”读取，只选择一段展示文本。
262          decision: Literal['通过'] | Literal['丢弃'] = "通过" if accepted else "丢弃"
263          record(events, f"过滤: {document['document_id']} -> {decision}")
264          if accepted:
265              # 通过时转交原来的 Document 字典引用，不复制；丢弃时循环继续向上游取下一项。
266              yield document
267
268
269  # 转换器仍然是一进一出的惰性阶段，只在下游索取时才构造对应 SearchHit。
270  def transform_documents(documents: Iterable[Document], events: list[str]) -> Iterator[SearchHit]:
271      for document in documents:
272          # 受控错误放在真正构造结果之前，用来模拟“前几项成功，后续某项转换失败”。
273          if document["document_id"] == "doc-error":
274              record(events, "转换: doc-error 抛出受控异常")
275              raise DocumentTransformError("doc-error 无法转换")
```

```python

# 转换器仍然是一进一出的惰性阶段，只在下游索取时才构造对应 SearchHit。
def transform_documents(documents: Iterable[Document], events: list[str]) -> Iterator[SearchHit]:
    for document in documents:
        # 受控错误放在真正构造结果之前，用来模拟“前几项成功，后续某项转换失败”。
        if document["document_id"] == "doc-error":
            record(events, "转换: doc-error 抛出受控异常")
            raise DocumentTransformError("doc-error 无法转换")

        # 这里新建一个 SearchHit 字典，不修改上游 Document。
        # f-string 的 :.2f 格式规格把分数显示为固定两位小数，例如 0.95 -> "0.95"。
        hit: SearchHit = {
            "document_id": document["document_id"],
            "label": f"{document['title']} ({document['score']:.2f})",
        }
        record(events, f"转换: yield {hit['document_id']}")
        # 只有这个结果被请求时才 yield，因此错误文档之后的记录不会被提前转换。
        yield hit


# 这个限量器通过显式 next() 精确控制拉取次数，而不是先取得第 limit + 1 项再判断停止。
def take_hits(hits: Iterable[SearchHit], limit: int, events: list[str]) -> Iterator[SearchHit]:
    """最多拉取 limit 项，不为判断停止而多拉取第 limit + 1 项。"""
    if limit < 0:
        # 因为本函数含 yield，调用只创建生成器；这个检查也要等首次消费时才执行。
        raise ValueError("limit 不能小于 0")

    # 统一取得一个迭代器并保存其位置。若 hits 已是迭代器，iter(hits) 通常仍返回它自己。
    iterator: Iterator[SearchHit] = iter(hits)
    # range(1, limit + 1) 产生 1 到 limit；limit 为 0 时循环零次，上游一次也不会被拉取。
    for position in range(1, limit + 1):
        try:
            # 每轮只在这里向上游请求一项，所以成功次数最多就是 limit。
            hit: SearchHit = next(iterator)
        except StopIteration:
            # 上游不足 limit 项属于正常结束；生成器中的 return 会让下游也收到 StopIteration。
            record(events, "限量: 上游已经耗尽")
```

```python
# 这个限量器通过显式 next() 精确控制拉取次数，而不是先取得第 limit + 1 项再判断停止。
def take_hits(hits: Iterable[SearchHit], limit: int, events: list[str]) -> Iterator[SearchHit]:
    """最多拉取 limit 项，不为判断停止而多拉取第 limit + 1 项。"""
    if limit < 0:
        # 因为本函数含 yield，调用只创建生成器；这个检查也要等首次消费时才执行。
        raise ValueError("limit 不能小于 0")

    # 统一取得一个迭代器并保存其位置。若 hits 已是迭代器，iter(hits).通常仍返回它自己。
    iterator: Iterator[SearchHit] = iter(hits)
    # range(1, limit + 1) 产生 1 到 limit；limit 为 0 时循环零次，上游一次也不会被拉取。
    for position in range(1, limit + 1):
        try:
            # 每轮只在这里向上游请求一项，所以成功次数最多就是 limit。
            hit: SearchHit = next(iterator)
        except StopIteration:
            # 上游不足 limit 项属于正常结束；生成器中的 return 会让下游也收到 StopIteration。
            record(events, "限量: 上游已经耗尽")
            return
        record(events, f"限量: 放行第 {position} 条 {hit['document_id']}")
        yield hit

    # 完成恰好 limit 轮后直接结束，不再用一次多余 next() 探测上游是否还有数据。
    record(events, f"限量: 已达到 {limit} 条，不再请求上游")


# build_pipeline 本身没有 yield，是普通函数；调用时会执行组装代码并立即返回最外层生成器。
# 各阶段函数都含 yield，所以这里创建它们并不会遍历 documents，也不会打开文件源。
def build_pipeline(
    documents: Iterable[Document],
    *,
    limit: int,
    events: list[str],
) -> Iterator[SearchHit]:
    """这里只组装惰性阶段，不消费 documents。"""
    # 第一层只保存筛选规则及上游引用；关键字参数让两个业务条件清晰可见。
    filtered: Iterator[Document] = filter_documents(
```

```python

# build_pipeline 本身没有 yield，是普通函数；调用时会执行组装代码并立即返回最外层生成器。
# 各阶段函数都含 yield，所以这里创建它们并不会遍历 documents，也不会打开文件源。
def build_pipeline(
    documents: Iterable[Document],
    *,
    limit: int,
    events: list[str],
) -> Iterator[SearchHit]:
    """这里只组装惰性阶段，不消费 documents。"""
    # 第一层只保存筛选规则及上游引用；关键字参数让两个业务条件清晰可见。
    filtered: Iterator[Document] = filter_documents(
        documents,
        category="python",
        minimum_score=0.8,
        events=events,
    )
    # 转换器把 filtered 作为自己的上游，限量器再包住转换器，形成一条相互引用的惰性链。
    transformed: Iterator[SearchHit] = transform_documents(filtered, events)
    # 返回的是尚未启动的 take_hits 生成器；真正的数据流动由后续消费者的 next()/for 触发。
    return take_hits(transformed, limit, events)


# 这是明确的“物化边界”：把逐项到达的 SearchHit 保存成可重复读取的 list。
# 函数不捕获上游异常；若消费失败，异常会传播给调用方，局部 results 不会作为返回值交付。
def consume_hits(hits: Iterable[SearchHit], events: list[str]) -> list[SearchHit]:
    """消费者决定何时把惰性流物化为 list。"""
    results: list[SearchHit] = []
    # for 会一直拉取直到上游 StopIteration；每项先记录消费事件，再追加同一字典引用。
    for hit in hits:
        record(events, f"消费者: 收到 {hit['document_id']}")
        results.append(hit)
    # 只有上游自然结束时才运行到这里并返回完整结果。
    return results


```

```
347
348 def demonstrate_iteration_protocol() -> None:
349     print_section("1. iterable、iterator、next 与 StopIteration")
350     # list 是保存元素的可迭代容器。每次 iter(container) 都创建一条从开头出发的新迭代路径，
351     # 所以下面两个对象互不共享当前位置。
352     container: list[str] = ["A", "B", "C"]
353     first_iterator: Iterator[str] = iter(container)
354     second_iterator: Iterator[str] = iter(container)
355
356     # is 比较两个名字是否指向同一对象，不是比较内容；这里预期打印 True。
357     print("  list 两次 iter 得到不同迭代器:", first_iterator is not second_iterator)
358     # list(container) 内部各自创建并耗尽一个新迭代器，不会消耗 container 本身保存的元素，
359     # 所以两次都能得到 ["A", "B", "C"]。
360     print("  容器第一次遍历:", list(container))
361     print("  容器第二次遍历:", list(container))
362
363     # 赋值不会复制迭代器。shared_iterator 和 alias 是同一个对象的两个名字，位置完全共享。
364     shared_iterator: Iterator[str] = iter(container)
365     alias: Iterator[str] = shared_iterator
366     # 迭代器的 iter() 返回它自己；alias 取走 "A" 后，另一个名字也只能从 "B" 继续。
367     print("  iter(iterator) 仍是自己:", iter(shared_iterator) is shared_iterator)
368     print("  alias 取走一个值:", next(alias))
369     print("  原名字只能看到剩余值:", list(shared_iterator))
370     # 上一行的 list(...) 已持续消费到 StopIteration，同一个对象不会自动复位。
371     print("  同一个迭代器第二次已经为空:", list(alias))
372
373     # 单元素 tuple 必须写成 ("唯一值",)，结尾逗号才定义 tuple；圆括号本身只是分组。
374     exhausted: Iterator[Literal['唯一值']] = iter(("唯一值",))
375     print("  next 取得:", next(exhausted))
376     try:
377         # 唯一值已被取走，再调用 next() 会按协议抛 StopIteration。
378         next(exhausted)
379     except StopIteration:
380         print("  再次 next: 捕获 StopIteration")
381     # next(iterator, default) 把耗尽转换为默认返回值，适合不想写 try/except 的探测场景。
382     print("  next(iterator, default):", next(exhausted, "<已耗尽>"))
383
```

```python
def demonstrate_iteration_protocol() -> None:
    # tuple 也是 Iterable[str]；辅助函数会展示 for 在解释器内部依赖的核心协议。
    print("  手写 for 的核心过程，结果:", consume_like_for(("甲", "乙")))


def demonstrate_generator_state() -> None:
    print_section("2. yield 暂停并保留局部状态")
    # events 从空 list 开始。调用生成器函数只得到 source 对象，尚未进入 counted_source 函数体，
    # 所以下一行观察到的长度应是 0。
    events: list[str] = []
    source: Iterator[int] = counted_source(3, events)
    print("  调用生成器函数后事件数:", len(events))

    # 第一次 next() 从函数开头运行到 yield 1；第二次先从 yield 后恢复，再运行到 yield 2。
    print("  第一次 next 返回:", next(source))
    # 生成器表达式逐条检查事件文本，startswith("生产") 为每条消息产生 bool；
    # bool 可按 1/0 被 sum() 相加，因此结果就是已经出现的生产事件数量。
    print("  此时生产事件数:", sum(message.startswith("生产") for message in events))
    print("  第二次 next 返回:", next(source))
    # list(source) 从当前位置继续，而不是重新调用 counted_source；这里只会收集尚未产出的值 3。
    print("  list(source) 消费剩余值:", list(source))
    # 生成器耗尽后会稳定保持耗尽状态，带默认值的 next() 因而返回展示用字符串。
    print("  耗尽后带默认值的 next:", next(source, "<已耗尽>"))

    # 重新调用生成器函数才会得到一条全新的迭代路径和新的局部状态。
    batch_events: list[str] = []
    print("\n  新生成器交给 list(...): ")
    # 这次 list() 一次拉取到自然耗尽，因此三个 yield 值都会被立即物化。
    all_values: list[int] = list(counted_source(3, batch_events))
    print("  list 一次消费得到:", all_values)
    print("  list 调用期间生产数量:", sum(message.startswith("生产") for message in batch_events))


def demonstrate_comprehensions() -> None:
    print_section("3. 列表推导式与生成器表达式")
    # 列表推导式按“对右侧 iterable 的每个 value，计算左侧表达式并收集”来读。
```

```python

def demonstrate_comprehensions() -> None:
    print_section("3. 列表推导式与生成器表达式")
    # 列表推导式按“对右侧 iterable 的每个 value，计算左侧表达式并收集”来读。
    # 创建 squares 的这一行就会计算三次，并把三个整数全部保存进 list。
    list_events: list[str] = []
    squares: list[int] = [trace_square(value, list_events) for value in (1, 2, 3)]
    print("  列表推导式创建完成，结果:", squares)

    expression_events: list[str] = []
    # 生成器表达式不会立刻计算 trace_square，但最左侧 for 的 iterable 表达式
    # build_eager_iterable(...) 会立即调用，并对返回的 list 立即取得 iterator。
    lazy_squares: Generator[int, None, None] = (
        trace_square(value, expression_events) for value in build_eager_iterable(expression_events)
    )
    # 因而此时只有“创建 list”事件，没有任何平方计算事件。
    print("  生成器表达式刚创建时的事件:", expression_events)
    # 第一次 next() 只计算 1 的平方；list() 随后从当前位置计算 2、3 并耗尽表达式。
    print("  next 只计算一项:", next(lazy_squares))
    print("  list(...) 再计算剩余项:", list(lazy_squares))
    # lazy_squares 是迭代器而不是可重复遍历的结果容器，耗尽后再次 list() 得到空列表。
    print("  同一表达式再次遍历:", list(lazy_squares))

    stream_events: list[str] = []
    # 这里最左侧调用 counted_source(...) 的确立即发生，但它是生成器函数：调用只返回对象，
    # 函数体依然没有启动，所以创建 fully_lazy 后 events 仍为空。
    fully_lazy: Generator[int, None, None] = (trace_square(value, stream_events) for value in counted_source(2, stre
    print("  最左侧本身是生成器时，创建后的事件:", stream_events)
    # 首次 next(fully_lazy) 会先启动上游产出 1，再把 1 交给 trace_square，最后返回平方结果 1。
    print("  首次消费后得到:", next(fully_lazy))


def demonstrate_yield_return_and_delegation() -> None:
    print_section("4. yield、return 与 yield from")
    return_events: list[str] = []
    # 创建时函数体未运行；第一次 next() 才记录事件并取得唯一的 yield 值“数据块”。
```

```
447 def demonstrate_yield_return_and_delegation() -> None:
448     print_section("4. yield、return 与 yield from")
449     return_events: list[str] = []
450     # 创建时函数体未运行；第一次 next() 才记录事件并取得唯一的 yield 值“数据块”。
451     generator: Generator[str, None, str] = one_chunk_then_return(return_events)
452     print("  第一次 next 得到 yield 值:", next(generator))
453     try:
454         # 第二次恢复会执行 return "完成值"，协议通过 StopIteration 结束迭代。
455         next(generator)
456     except StopIteration as completion:
457         # `as completion` 把异常对象绑定到变量；return 的参数保存在它的 value 属性中。
458         # 普通 for/list 会自行吞掉 StopIteration，因此只有这种手动消费才能直接观察完成值。
459         print("  return 没有再产出一项；完成值是:", completion.value)
460
461     delegation_events: list[str] = []
462     # combine_sources 也是生成器函数，创建 combined 时还未记录“组合器开始”。
463     combined: Iterator[str] = combine_sources(delegation_events)
464     print("  组合生成器刚创建时事件数:", len(delegation_events))
465     # list() 负责拉到耗尽，最终顺序是组合器自己的开头、本地两项、缓存一项。
466     print("  消费组合结果:", list(combined))
467
468
469 def demonstrate_file_pipeline() -> None:
470     print_section("5. JSONL -> 过滤 -> 转换 -> 最多消费 N 条")
471     events: list[str] = []
472     # source、pipeline 都只是尚未启动的生成器对象。build_pipeline 仅连接各阶段的引用，
473     # 所以到 print 时 events 仍为空，documents.jsonl 也尚未打开。
474     source: Generator[Document, None, None] = load_jsonl(DATA_PATH, events)
475     pipeline: Iterator[SearchHit] = build_pipeline(source, limit=2, events=events)
476     print("  流水线组装后事件数（文件尚未打开）:", len(events))
477
478     # 先绑定空 list，使 finally 之后仍有一个定义好的 results 可供打印和断言。
479     results: list[SearchHit] = []
480     try:
481         # consume_hits 的 for 从最下游开始拉取；被过滤的行不会计入 limit，但确实已经从文件读取。
482         results = consume_hits(pipeline, events)
```

```
469 def demonstrate_file_pipeline() -> None:
478     # 先绑定空 list，使 finally 之后仍有一个定义好的 results 可供打印和断言。
479     results: list[SearchHit] = []
480     try:
481         # consume_hits 的 for 从最下游开始拉取；被过滤的行不会计入 limit，但确实已经从文件读取。
482         results = consume_hits(pipeline, events)
483         # limit=2 会让限量器主动停止，此时文件源仍暂停在第三行的 yield，尚未自然离开 with。
484         print("  达到上限后、显式 close 前已关闭:", CLOSED_EVENT in events)
485     finally:
486         # take_hits 达到上限时上游仍可继续；创建并持有真正文件源的代码负责结束它。
487         # close() 会在源暂停处触发清理，使 load_jsonl 的 with 和 finally 都确定执行。
488         source.close()
489
490     # 生成器表达式只筛选以固定前缀开头的事件，sum(bool) 得出真实文件读取次数。
491     read_count: int = sum(message.startswith("源: 读取第") for message in events)
492     print("  最终结果:", results)
493     print("  实际只读取行数:", read_count)
494     print("  close 后已关闭:", CLOSED_EVENT in events)
495
496     # 列表推导式从两条 SearchHit 中抽取 id；验证第一、三行通过筛选且顺序不变。
497     assert [hit["document_id"] for hit in results] == ["doc-001", "doc-003"]
498     # 为得到两条合格结果只需读三行；第四行 doc-error 不应被限量器多拉取。
499     assert read_count == 3
500     # any() 会短路检查事件流；结果为 False 证明连相关日志都没有出现，即错误文档未被触碰。
501     assert not any("doc-error" in message for message in events)
502
503
504 def demonstrate_boundaries_and_failure() -> None:
505     print_section("6. 空输入、自然耗尽、中途异常与提前停止")
506
507     print("\n  -- 空输入 --")
508     empty_events: list[str] = []
509     # () 是空 tuple，可作为不会产出任何 Document 的 iterable；首次拉取会沿各层传播耗尽信号。
510     empty_results: list[SearchHit] = consume_hits(build_pipeline((), limit=3, events=empty_events), empty_events)
511     print("  空输入结果:", empty_results)
512     assert empty_results == []
```

```python
print("\n  -- 上游少于 N 条，自然耗尽 --")
# tuple[Document, ...] 中的 ... 表示可以有任意多个 Document 元素。
# 外层 tuple 不可变，但内部两个 dict 仍是可变对象；本例只读取它们，不做修改。
safe_documents: tuple[Document, ...] = (
    {
        "document_id": "memory-1",
        "title": "内存文档一",
        "category": "python",
        "score": 0.91,
    },
    {
        "document_id": "memory-2",
        "title": "内存文档二",
        "category": "python",
        "score": 0.89,
    },
)
exhaustion_events: list[str] = []
# limit=5 大于实际两项，因此不是限量器主动截断，而是上游 tuple 先自然耗尽。
exhausted_pipeline: Iterator[SearchHit] = build_pipeline(safe_documents, limit=5, events=exhaustion_events)
exhausted_results: list[SearchHit] = consume_hits(exhausted_pipeline, exhaustion_events)
print("  自然耗尽结果数:", len(exhausted_results))
# exhausted_pipeline 是同一个已经耗尽的生成器对象，再次 list() 不会重跑 safe_documents。
print("  同一流水线再次遍历:", list(exhausted_pipeline))
assert len(exhausted_results) == 2

print("\n  -- 消费到中途才抛异常，先前结果仍已交付 --")
failure_events: list[str] = []
# limit=10 足够大，会继续读取到第四行 doc-error，而不是像上一实验那样提前停止。
failing_source: Generator[Document, None, None] = load_jsonl(DATA_PATH, failure_events)
failing_pipeline: Iterator[SearchHit] = build_pipeline(failing_source, limit=10, events=failure_events)
# 这里不用 results = list(...)，而是逐项 append；这样异常发生时仍能观察已交付的部分结果。
partial_results: list[SearchHit] = []
# 先给异常消息一个稳定默认值，便于 try/except 之后做精确断言。
caught_message: Literal[''] = ""
```

```
504  def demonstrate_boundaries_and_failure() -> None:
547      # 先给异常消息一个稳定默认值，便于 try/except 之后做精确断言。
548      caught_message: Literal[''] = ""
549      try:
550          # 前两条合格 hit 会先完成记录和 append；迭代到 doc-error 时才跳出 for 进入 except。
551          for hit in failing_pipeline:
552              record(failure_events, f"消费者: 收到 {hit['document_id']}")
553              partial_results.append(hit)
554      except DocumentTransformError as error:
555          # 只捕获本实验预期的转换异常，其他异常不会被误报成同一种业务失败。
556          caught_message = str(error)
557          print("  消费者捕获:", type(error).__name__, caught_message)
558      finally:
559          # 无论是否得到预期异常，都显式关闭仍可能暂停的文件源。
560          failing_source.close()
561
562      # 未处理异常从生成器内部逸出后，这条 failing_pipeline 已终止，不能从失败点继续。
563      print("  异常前已收到:", [hit["document_id"] for hit in partial_results])
564      print("  异常后流水线已耗尽:", list(failing_pipeline))
565      print("  异常路径文件已关闭:", CLOSED_EVENT in failure_events)
566      # 三个断言分别验证部分结果、具体错误内容和资源清理，避免只检查“确实抛错”。
567      assert [hit["document_id"] for hit in partial_results] == ["doc-001", "doc-003"]
568      assert caught_message == "doc-error 无法转换"
569      assert CLOSED_EVENT in failure_events
570
571      print("\n  -- 消费者提前停止时，break 本身不替持有者关闭源 --")
572      early_events: list[str] = []
573      early_source: Generator[Document, None, None] = load_jsonl(DATA_PATH, early_events)
574      early_pipeline: Iterator[SearchHit] = build_pipeline(early_source, limit=10, events=early_events)
575      # 只调用一次 next()，沿整条链拿到第一条合格结果后就停下；源此时暂停在自己的 yield。
576      first_hit: SearchHit = next(early_pipeline)
577      record(early_events, f"消费者: 只要第一条 {first_hit['document_id']}，随后停止")
578      # 仍持有 early_pipeline/early_source 引用，不能依赖对象回收时机自动关闭文件。
579      print("  停止消费后、显式 close 前已关闭:", CLOSED_EVENT in early_events)
580      early_source.close()
581      print("  显式 close 后已关闭:", CLOSED_EVENT in early_events)
```

```
504 def demonstrate_boundaries_and_failure() -> None:
579     print("  停止消费后、显式 close 前已关闭:", CLOSED_EVENT in early_events)
580     early_source.close()
581     print("  显式 close 后已关闭:", CLOSED_EVENT in early_events)
582     # 外层流水线对象本身尚未直接 close，但它再次向已关闭的源拉取时只会收到耗尽信号。
583     print("  上游关闭后流水线剩余结果:", list(early_pipeline))
584     assert CLOSED_EVENT in early_events
585
586
587 # main 只负责安排演示顺序；上面的 def 在导入模块时创建函数对象，但不会自动执行函数体。
588 def main() -> None:
589     demonstrate_iteration_protocol()
590     demonstrate_generator_state()
591     demonstrate_comprehensions()
592     demonstrate_yield_return_and_delegation()
593     demonstrate_file_pipeline()
594     demonstrate_boundaries_and_failure()
595     print("\n所有第 17 章示例与运行时断言均已完成。")
596
597
598 # Python 直接运行此文件时，内置变量 __name__ 为 "__main__"，于是调用 main()。
599 # 其他模块若只 import 本文件，条件为 False，便能复用函数而不自动打印或读取数据文件。
600 if __name__ == "__main__":
601     main()
602
```